

CHECK WHAT YOU LEARN'T

Check Yourself - Unit 1**A. Choose the correct answer**

1. A robot must be able to sense, think and _____.
(a) sleep (b) act (c) talk (d) fly
2. Electricity flows only when the circuit is
(a) broken (b) hot (c) a complete loop (d) made of plastic
3. The job of a 220 ohm resistor next to an LED is to
(a) make it brighter (b) protect it (c) change its colour (d) hold it in place
4. In the middle of a breadboard, holes are joined in
(a) columns of five (b) rows of ten (c) no pattern (d) pairs
5. The long leg of an LED must be connected towards
(a) GND (b) the minus side (c) the plus side (d) either side
6. Which of these is measured in ohms?
(a) voltage (b) current (c) resistance (d) time

B. Fill in the blanks

- 1 The battery pushes electricity along. That push is called _____.
- 2 A switch works by _____ the circuit on purpose.
- 3 The two long side rails of a breadboard are used for _____ and ground.
- 4 LED is short for Light _____ Diode.

C. Answer in one or two lines

- 1 Name one machine at home that follows Sense - Think - Act, and explain each step.
- 2 Your LED does not glow. Write the first three things you would check, in order.
- 3 Why can you never join 5V straight to GND with a plain wire?
- 4 Explain in your own words why a resistor is needed next to an LED.

UNIT 2

The Arduino IDE

Sessions 3, 4 and 5

Now the board gets a brain transplant - yours. You will meet the Arduino Uno properly, install the software, and send your very first program down the USB cable into a real piece of hardware.

By the end of this unit you will be able to:

- Point to every important part of the Arduino Uno board.
- Select the right board and port and upload a program successfully.
- Explain what `setup()` and `loop()` do, and why the difference matters.
- Write your own code that switches LEDs on and off in a chosen order.

SESSION 3

one class

Meet the Board and the Software

Today you will

- Learn the parts of the Arduino Uno board.
- Install the Arduino IDE and set the board and port.
- Upload the Blink program and watch a real LED obey you.

The Arduino Uno is a small computer with no screen and no keyboard. It cannot run apps. It runs exactly one program - the one you give it - over and over, forever. It keeps that program in memory even after the power is pulled out, and starts running it again the instant power returns. That is why a machine built on an Arduino still works when you unplug it from the laptop and power it from a battery.

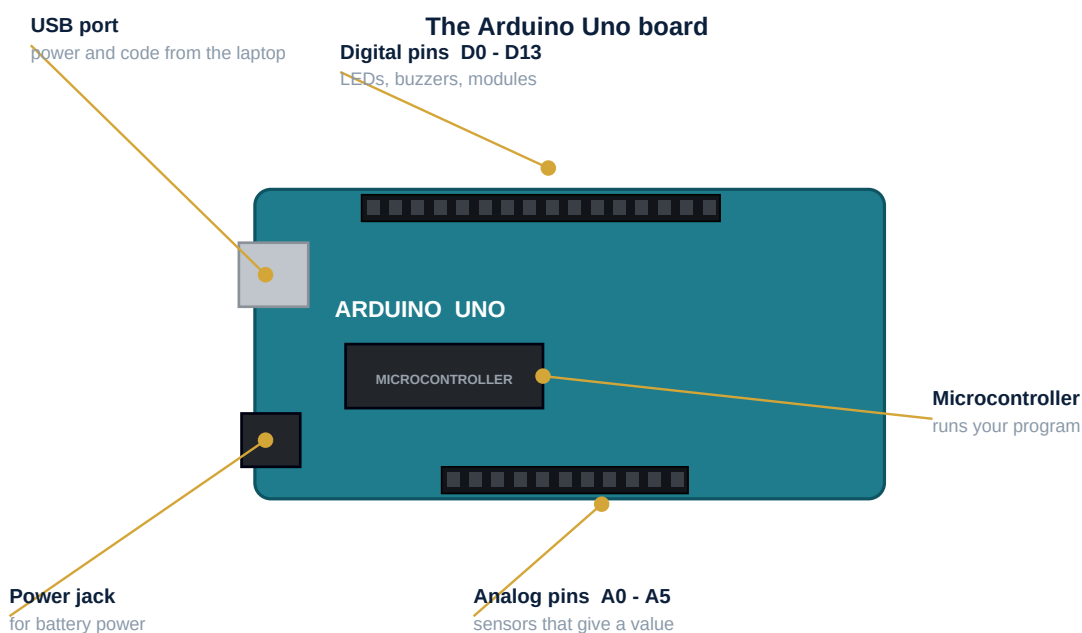


Figure 2.1 - Know your board before you wire anything to it

Part of the board	What you use it for
Digital pins D0 to D13	Anything that is simply ON or OFF: LEDs, buzzers, modules.
Analog pins A0 to A5	Sensors that give a value rather than just on or off.
5V and GND pins	Power out to your components. Every circuit needs both of these.
USB port	Sends your program in from the laptop, and powers the board.
Power jack	For running your finished machine from a battery instead.
Reset button	Restarts your program from the very beginning.
Pins marked with ~	Can fade an output smoothly instead of only switching it.

3.1 The Software: Arduino IDE

IDE stands for Integrated Development Environment, which is a very long name for the window where you type your program. Ignore most of the buttons for now. Two of them matter more than all the rest: **Verify** checks your code for mistakes without sending it anywhere, and **Upload** sends the checked code into the board.

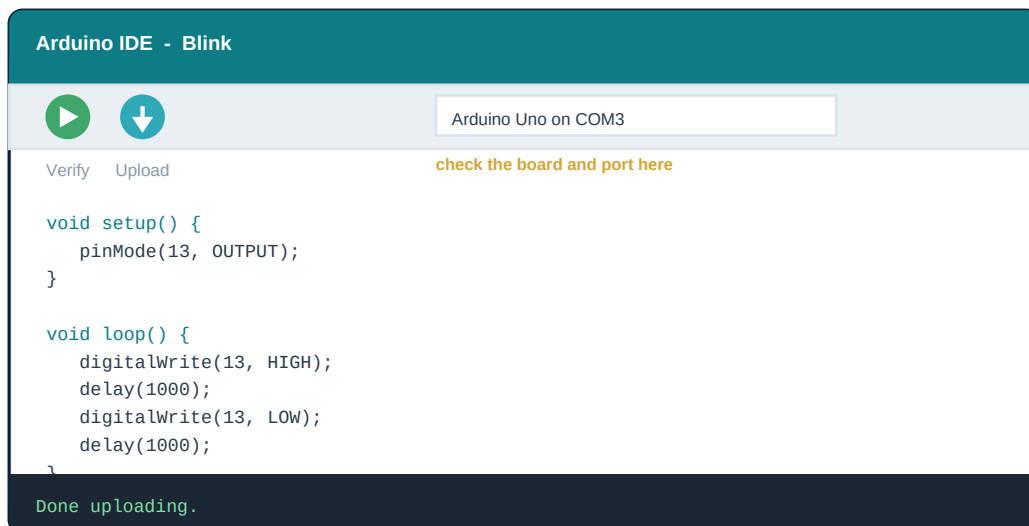


Figure 2.2 - Verify checks your code; Upload sends it to the board

Watch Out

Before you press Upload for the very first time, check two settings. **Tools > Board** must say Arduino Uno, and **Tools > Port** must have a port selected. When an upload fails, this is the first thing to check - every single time, for the rest of your life.

3.2 Your First Program: Blink

Every Arduino learner on the planet starts at exactly the same place. Blink switches an LED on for one second, off for one second, and repeats forever. Build the circuit first, then type the code.

D13 -> 220 ohm resistor -> LED -> GND

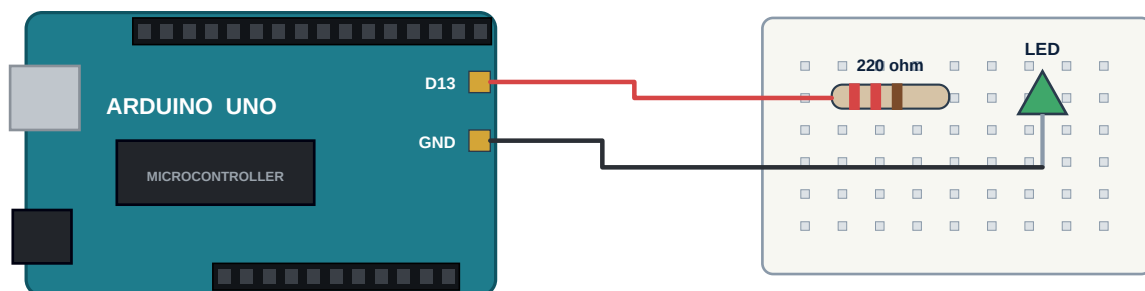


Figure 2.3 - Your first circuit that a program controls

Blink.ino

```
int led = 13;           // the pin our LED is connected to

void setup() {
  pinMode(led, OUTPUT); // this pin will send power out
}

void loop() {
  digitalWrite(led, HIGH); // switch the LED on
  delay(1000);             // wait 1000 milliseconds = 1 second
  digitalWrite(led, LOW);  // switch the LED off
}
```

```
delay(1000);           // wait again
}
```

Press Verify. If the bar at the bottom turns green and says *Done compiling*, your grammar is correct. Now press Upload. The two small lights near the USB socket will flicker as your program travels down the cable. A second later, your LED begins to blink - and it is doing that because of a sentence you wrote.

SESSION 4

one class

The Grammar of Arduino Code

Today you will

- Learn what every word in your program actually means.
- Understand the difference between `setup()` and `loop()`.
- Read an error message and work out what it is telling you.

Arduino code looks frightening for about ten minutes, and then it looks obvious. That is because there are only a handful of words to learn, and each one does exactly what its name suggests. Here is the whole vocabulary you need for Units 1 to 4.

What you type	What it really means
<code>void setup()</code>	Do everything inside these brackets once , the moment the board wakes up.
<code>void loop()</code>	Do everything inside these brackets again and again , forever, until the power is cut.
<code>pinMode(9, OUTPUT)</code>	Pin 9 is going to send power out to something.
<code>pinMode(2, INPUT)</code>	Pin 2 is going to listen to something instead.
<code>digitalWrite(9, HIGH)</code>	Send 5V out of pin 9 - switch it on.
<code>digitalWrite(9, LOW)</code>	Send 0V out of pin 9 - switch it off.
<code>delay(1000)</code>	Freeze here for 1000 milliseconds, then carry on.
<code>int led = 13;</code>	Give the number 13 a name, so the code reads like English.
<code>// like this</code>	A comment. The board ignores it completely; humans read it.
<code>;</code> at the end of a line	A full stop. Almost every instruction needs one.
<code>{ }</code>	Brackets that hold a group of instructions together. They always come in pairs.

The difference between `setup()` and `loop()` is worth one extra minute. Think of `setup()` as getting dressed in the morning - you do it once, and then you get on with your day. `loop()` is the day itself, repeating until you sleep. Telling the board *pin 9 is an output* belongs in `setup()`, because it does not need saying twice. Telling the board *switch pin 9 on* belongs in `loop()`, because you want it to keep happening.

Did You Know

`delay(1000)` means one thousand **milliseconds**, and a millisecond is one thousandth of a second. So `delay(1)` would blink the LED a thousand times a second - far too fast for your eye. It would simply look permanently half-bright.

4.1 Errors You Will Definitely Meet

Error messages look unfriendly, but they are trying to help. The trick is that the error is often reported on the line *after* the real mistake. Learn these five and you will solve most of your own problems without help.

What the screen says	What is actually wrong	How to fix it
expected ';' before ...	You forgot a semicolon	Add <code>;</code> at the end of the line above

What the screen says	What is actually wrong	How to fix it
'digitalwrite' was not declared	Wrong capital letters	Arduino is fussy: digitalWrite, not digitalwrite
expected '}' at end of input	A missing closing bracket	Count your { and } - they must pair up exactly
Board at COM__ is not available	Wrong port, or a charging-only cable	Re-select Tools > Port, or try a different USB cable
Code uploads but nothing happens	The fault is in the wiring, not the code	Check LED direction, the resistor and the GND wire

Try This

Deliberately break your working Blink program. Delete one semicolon and press Verify. Read the message carefully, then put the semicolon back. Doing this on purpose once means the message will never frighten you again.

SESSION 5

one class

Three LEDs, One Board

Today you will

- Wire three LEDs, each with its own resistor.
- Write a chase light that runs across all three.
- Take on the SOS challenge using only delay and digitalWrite.

One LED proves your code works. Three LEDs prove you can control **order** and **timing** - and order and timing are the entire idea behind your first project. Wire an LED with a 220 ohm resistor to pin D8, another to D9, and a third to D10. All three short legs go to the same GND rail on the breadboard.

ChaseLight.ino

```
int a = 8;
int b = 9;
int c = 10;

void setup() {
  pinMode(a, OUTPUT);
  pinMode(b, OUTPUT);
  pinMode(c, OUTPUT);
}

void loop() {
  digitalWrite(a, HIGH); delay(200); digitalWrite(a, LOW);
  digitalWrite(b, HIGH); delay(200); digitalWrite(b, LOW);
  digitalWrite(c, HIGH); delay(200); digitalWrite(c, LOW);
}
```

Notice that three instructions have been written on one line, separated by semicolons. The board does not care about lines at all - only about semicolons and brackets. Humans care about lines, which is why we usually keep one instruction per line. Here, keeping each LED on its own line makes the pattern easier to read.

Try This

Change every 200 to 60 and upload again. Then change them all to 800. You have just discovered that the same circuit can behave completely differently with no rewiring at all. That is the power of software.

Challenge

Morse code for SOS is three short flashes, three long flashes, three short flashes. Using only one LED, `delay()` and `digitalWrite()`, write the SOS signal. Use 200 ms for short, 600 ms for long, 200 ms for the gap between flashes and 1000 ms before the message repeats.

New word	What it means
Upload	Sending your program from the laptop into the board.
Comment	A note in the code for humans. The board ignores it.
Millisecond	One thousandth of a second.
Compile	Turning your typed words into instructions the chip understands.